

Sistema de SLAM, Navegación Autónoma y Búsqueda de Objetivos en un Robot Móvil

Martín Bianchi, Joaquín Szterensus, Juan Cruz Giner Pulero y Valentino Arbelaiz Alberti

Departamento de Ingeniería

Universidad de San Andrés

Victoria, Argentina

martinbianchi@udesa.edu.ar | jszterensus@udesa.edu.ar

jginerpulero@udesa.edu.ar | varbelaizalberti@udesa.edu.ar

Resumen—Este trabajo presenta un sistema de navegación autónoma para un robot móvil en un entorno tipo laberinto, con una misión final de búsqueda visual de conos rojos sobre un TurtleBot4 real. La solución se organiza como módulos de ROS 2 alrededor de un mapa de ocupación: mapeo con Grid-Based FastSLAM, localización probabilística con filtro de partículas, planificación global con A* sobre un costmap inflado, seguimiento de trayectoria supervisado por una máquina de estados, y una capa de misión que convierte detecciones visuales en objetivos de navegación solo después de validarlos geoméricamente contra el mapa. El sistema se validó en tres entornos de realismo creciente: simulación con TurtleBot3, rosbags grabados con el TurtleBot4 en el laberinto físico, y el robot real en el laboratorio. En simulación se completaron el mapeo y la navegación, incluyendo la detención segura ante objetivos inalcanzables. El procesamiento offline permitió calibrar el detector de conos (sin falsos positivos revisados manualmente sobre 1798 frames) y reveló que el LIDAR del robot está montado rotado 90°. Sobre el robot real se generó el mapa del laberinto en vivo, se validó la navegación sobre ese mapa y se completó la misión visual del cono. La integración real permitió identificar diferencias sim-to-real no presentes en simulación: namespaces de tópicos, frames TF, orientación física del LIDAR, ajuste de inflación en pasillos angostos y una diferencia geométrica entre cámara y LIDAR. En particular, el cono podía quedar por debajo del plano del RPLIDAR: la cámara lo detectaba, pero el LIDAR medía la pared de fondo. Para resolverlo se incorporó un fallback de servoing visual con micro-goals validados contra el mapa inflado. La experiencia muestra que la validación offline con datos reales permite anticipar brechas de percepción y geometría, pero que el cierre end-to-end requiere pruebas sobre el robot físico.

Index Terms—robótica móvil, SLAM, localización probabilística, navegación autónoma, TurtleBot

I. INTRODUCCIÓN

Se desarrolló un sistema de navegación autónoma para un robot móvil en un entorno tipo laberinto. El sistema parte de lecturas de LIDAR, odometría y cámara; construye una representación del espacio; estima la pose del robot; planifica trayectorias seguras; y coordina una misión visual de búsqueda de conos rojos. La decisión de diseño principal fue usar un mapa de ocupación como representación común para localizar, planificar y validar objetivos.

El desarrollo se organizó en tres etapas técnicas. Primero se generó un mapa del entorno mediante localización y mapeo simultáneos (SLAM). Luego se construyó una pila de navegación que estima la pose del robot dentro de ese

mapa, calcula una trayectoria hacia un objetivo y la sigue con velocidades acotadas. Por último, se diseñó una capa de misión orientada al despliegue sobre TurtleBot4 real: la cámara propone detecciones de conos rojos, pero la navegación hacia un cono sólo se habilita después de estimar su posición y comprobar que el mapa permite una aproximación válida.

Esta separación es importante porque una detección visual no alcanza para decidir movimiento. Un cono puede aparecer en la imagen aunque esté detrás de una pared calada, una abertura o una zona no transitable. Por eso, la percepción se trata como una hipótesis; la misión la convierte en objetivo únicamente si supera una validación geométrica.

Las secciones previas al análisis describen el método implementado: arquitectura, mapeo, localización, navegación y diseño de misión. Los mapas finales, trayectorias, capturas de RViz y conclusiones experimentales se presentan únicamente en la sección de análisis.

II. OBJETIVOS

El objetivo general fue implementar una arquitectura de navegación autónoma capaz de cubrir el ciclo completo de mapeo, localización, planificación, seguimiento de trayectoria y misión visual orientada al despliegue sobre robot real.

En la etapa de mapeo, el objetivo fue construir una grilla de ocupación a partir de lecturas LIDAR y odometría. El mapa debía ser útil para las etapas posteriores: distinguir zonas libres, obstáculos y regiones desconocidas con suficiente calidad para que un planificador pudiera operar sobre él.

En la etapa de navegación, el objetivo fue permitir que el robot partiera de una pose inicial aproximada, recibiera un objetivo de destino y generara una trayectoria libre de colisiones. Durante la ejecución, el sistema debía seguir la trayectoria, ajustar la orientación final, aceptar nuevos objetivos y reaccionar ante obstáculos que no estuvieran presentes en el mapa original.

En la etapa de misión, el objetivo fue integrar percepción visual y navegación. La misión debía buscar conos rojos, descartar distractores cromáticos y validar geoméricamente el punto de aproximación antes de enviarlo al navegador. Esta validación es necesaria porque la cámara puede observar objetos que no son alcanzables en línea recta.

III. ARQUITECTURA GENERAL DEL SISTEMA

La arquitectura se organizó alrededor del mapa de ocupación. El módulo de mapeo produce esa representación; el localizador la usa para estimar la pose; el navegador la transforma en un mapa de costos (*costmap*) para planificar; y la misión la consulta antes de publicar un objetivo derivado de la percepción visual. La Fig. 1 muestra esta relación.

A nivel de implementación, cada bloque se corresponde con paquetes de Robot Operating System 2 (ROS 2), usando la organización por nodos, tópicos y lanzadores propia de la plataforma [3]. El mapeo se implementa en *maze_slam*; la localización y la navegación en *maze_nav*; la percepción visual en *maze_perception*; y la supervisión de misión en *maze_mission*. Estos nombres se incluyen como referencia técnica; la organización principal del sistema se entiende a partir de sus responsabilidades funcionales.

El código fuente de los paquetes implementados se entrega junto con el informe en el repositorio del proyecto [4].

IV. MAPEO DEL ENTORNO

IV-A. Enfoque elegido

El sistema usa una grilla de ocupación como representación principal del entorno. En esta representación, cada celda del mapa resume si una región del espacio parece libre, ocupada o desconocida. Esta elección conecta naturalmente con la navegación: la misma grilla puede inflarse para considerar el tamaño del robot, consultarse para validar objetivos y recorrerse con algoritmos de búsqueda.

Para construir la grilla se implementó Grid-Based FastSLAM, siguiendo el enfoque probabilístico de mapas de ocupación y filtros de partículas usado en robótica móvil [1]. El enfoque mantiene un conjunto de hipótesis de pose; cada partícula mantiene una hipótesis de pose y su propia estimación del mapa. A medida que llegan nuevas lecturas LIDAR, las hipótesis más consistentes con el entorno ganan peso y sus mapas se actualizan.

IV-B. Grid-Based FastSLAM

El ciclo de mapeo combina predicción por odometría y corrección con LIDAR. La odometría propone cómo se movió el robot desde la lectura anterior. Luego, cada partícula compara la nueva lectura LIDAR con su mapa mediante un campo de verosimilitud (*likelihood field*). Las partículas que explican mejor la lectura reciben mayor peso; después se actualiza la grilla usando los rayos del sensor. La Fig. 2 resume el flujo.

La ocupación de cada celda se actualiza en log-odds. La idea es sumar evidencia nueva sin descartar la acumulada: una lectura que atraviesa una celda refuerza la hipótesis de espacio libre, mientras que un impacto cercano refuerza la hipótesis de ocupación, como se muestra en la ecuación (1):

$$l_t(m_i) = l_{t-1}(m_i) + \log \left(\frac{p(m_i | z_t, x_t)}{1 - p(m_i | z_t, x_t)} \right) - l_0. \quad (1)$$

En esta expresión, $l_t(m_i)$ es el log-odds posterior de la celda m_i , z_t es la lectura LIDAR, x_t es la pose estimada y l_0 es

el prior. El mapa se guarda como un par de archivos *.pgm* y *.yaml*, formato que luego puede consumir la navegación.

IV-C. Exportación del mapa

El mapa exportado funciona como interfaz común entre módulos. El archivo de imagen codifica las celdas libres, ocupadas y desconocidas; el archivo de configuración conserva resolución, origen y umbrales de ocupación. De esta manera, la navegación no depende del estado interno del algoritmo de mapeo, sino de una representación persistente y revisable.

La evaluación visual del mapa y su utilidad para navegar se discuten en la sección de análisis. Antes de esa sección sólo se establece cómo se genera y cómo se conecta con el resto del sistema.

V. LOCALIZACIÓN Y NAVEGACIÓN AUTÓNOMA

V-A. Localización probabilística

Durante la navegación, la odometría no se toma como pose verdadera. Las ruedas acumulan error y ese error crece con el tiempo, por lo que el robot necesita corregirse usando el mapa y el LIDAR. Para eso se implementó Monte Carlo Localization (MCL), un filtro de partículas que mantiene varias hipótesis de pose y las pesa según qué tan bien explican las lecturas del sensor.

El usuario proporciona una pose inicial aproximada en RViz. A partir de esa estimación, el filtro distribuye partículas alrededor de la pose indicada. Con cada movimiento, las partículas se propagan con el modelo de odometría; con cada lectura LIDAR, se comparan contra el mapa mediante el campo de verosimilitud. La salida del filtro es una pose estimada y una nube de partículas útil para visualizar la incertidumbre de la localización.

V-B. Mapa de costos e inflación

El planificador no usa la grilla cruda directamente. Primero se construye un mapa de costos (*costmap*) donde las paredes, las celdas desconocidas y los obstáculos detectados se consideran regiones no transitables o costosas. Luego se aplica una inflación alrededor de los obstáculos para representar el tamaño del robot y un margen de seguridad.

La inflación cumple dos funciones. Por un lado, evita trayectorias pegadas a paredes. Por otro, permite que la misión use el mismo criterio para decidir si un punto de aproximación al cono es navegable. Si la misión publicara un objetivo dentro de una zona inflada, el navegador podría rechazarlo o no encontrar trayectoria; por eso ambos módulos deben usar criterios compatibles.

V-C. Planificación global con A*

El planificador global usa A* sobre una grilla 8-conectada, una estrategia clásica de búsqueda sobre espacios discretizados [2]. Cada transición tiene un costo geométrico y una penalización por cercanía a obstáculos. Así, si hay más de una ruta posible, se prefiere una trayectoria con mejor despeje lateral antes que una que pase innecesariamente cerca de paredes.

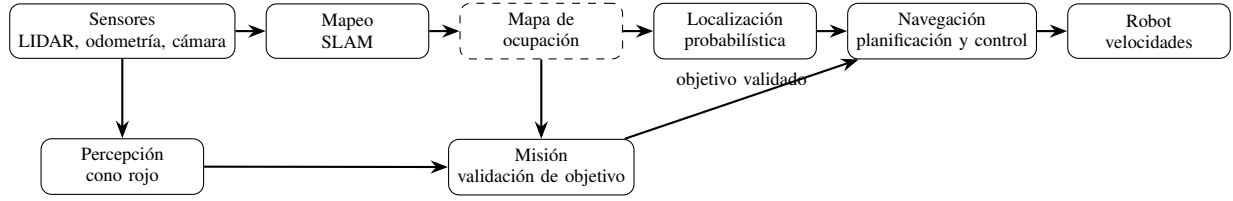


Figura 1. Arquitectura conceptual del sistema. El mapa de ocupación articula mapeo, localización, navegación y validación de objetivos; la misión decide qué objetivo enviar, pero no controla directamente las velocidades del robot.

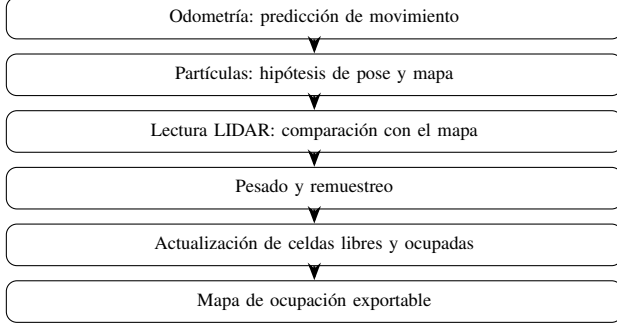


Figura 2. Flujo conceptual del mapeo mediante Grid-Based FastSLAM. La odometría predice el movimiento, el LIDAR corrige la consistencia de las partículas y la grilla acumula evidencia de ocupación.

El costo local combina el avance geométrico con una penalización por cercanía a obstáculos, como se muestra en la ecuación (2):

$$g(n') = g(n) + d(n, n') + \lambda \max(0, r + d_e - D(n')). \quad (2)$$

Aquí, $d(n, n')$ es la distancia entre celdas, $D(n')$ es la distancia de la celda candidata al obstáculo más cercano, r representa el margen mínimo asociado al robot y d_e agrega despeje extra. El peso λ regula cuánto se penalizan las celdas cercanas a obstáculos.

V-D. Seguimiento de trayectoria

Una trayectoria planificada todavía no mueve al robot por sí sola. El navegador debe convertirla en velocidades lineales y angulares. Para hacerlo, se toma un punto adelantado sobre la trayectoria y se calcula cómo orientar el robot hacia él. Si el error angular es grande, el robot gira primero; cuando la orientación es suficiente, avanza con velocidad acotada.

El seguimiento mantiene un índice monótono sobre la trayectoria para no volver a perseguir puntos ya superados. Al llegar cerca de la posición final, el sistema entra en una etapa separada de alineación, donde ajusta la orientación final pedida por el usuario.

V-E. Máquina de estados de navegación

La navegación se coordina con una máquina de estados finitos (finite state machine, FSM). Esta FSM separa espera de objetivo, planificación, seguimiento, alineación final, llegada y recuperación. La Fig. 3 muestra el flujo conceptual.

A nivel de ROS 2, el estado se publica para que otros módulos puedan observar si el navegador está planificando, siguiendo una trayectoria, recuperándose o si alcanzó el objetivo. La misión usa esa señal sólo como información de supervisión.

V-F. Obstáculos no mapeados

El mapa estático no puede cubrir todos los cambios del entorno. Durante el seguimiento, el navegador revisa las lecturas LIDAR frontales. Si aparece un obstáculo en una zona que el mapa consideraba libre, puede registrarse como obstáculo dinámico, reconstruirse el mapa de costos y volver a planificar.

Cuando el bloqueo persiste o no se encuentra una alternativa segura, el comportamiento preferido es detener el robot. Esta decisión prioriza declarar un bloqueo seguro antes que insistir sobre una zona no transitable.

VI. DISEÑO DE MISIÓN PARA ROBOT REAL

VI-A. Despliegue previsto sobre TurtleBot4

La etapa real se diseñó para ejecutar la misma cadena conceptual usada en simulación: mapa, localización, planificación, seguimiento y supervisión de misión. El TurtleBot4 agrega diferencias prácticas, como nombres de sensores asociados al robot, frames de odometría reales, configuración física del LIDAR y velocidades más conservadoras. Esas diferencias se resuelven mediante parámetros y perfiles de lanzamiento, sin cambiar la lógica principal de navegación.

En esta sección se describe el diseño previsto para el despliegue. La evidencia de funcionamiento en laboratorio, junto con sus limitaciones, corresponde al análisis de resultados.

VI-B. Percepción del cono rojo

El módulo de percepción procesa la imagen de cámara para proponer detecciones de conos rojos. Primero segmenta regiones en HSV; luego filtra blobs por tamaño y forma; finalmente entrega centroide, área, dirección aproximada y confianza. La detección no decide movimiento: sólo produce una hipótesis visual.

Esa hipótesis puede corresponder a un cono alcanzable, a un distractor o a un objeto visto a través de una pared calada. Por eso, la misión no navega directamente hacia toda detección roja. Antes debe estimar una posición en el mapa y comprobar que exista una aproximación navegable.

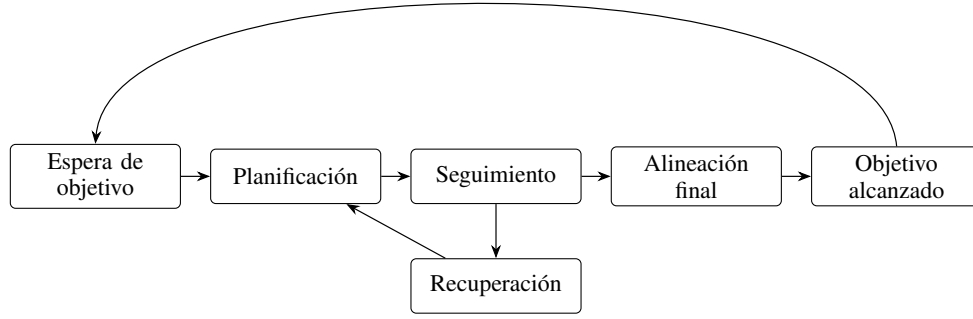


Figura 3. FSM de navegación. El flujo principal espera un objetivo, planifica, sigue la trayectoria y ajusta la orientación final; ante un bloqueo, pasa por recuperación antes de volver a planificar.

VI-C. Estimación métrica y validación geométrica

Para estimar la posición del cono, la misión combina la dirección obtenida desde la cámara con una distancia medida por LIDAR. Con la pose estimada del robot, esa medición puede proyectarse al marco del mapa.

La validación se pensó en dos niveles. Primero se revisa el mapa crudo: si el punto estimado del cono cae sobre una celda ocupada, la detección se interpreta como no navegable de forma directa y se descarta como objetivo inmediato. Luego se calcula un punto de aproximación separado del cono por una distancia de seguridad y se valida contra el mapa inflado. Si ese punto no es navegable, el navegador podría rechazarlo o no encontrar trayectoria.

Durante la integración real se agregó además un mecanismo de respaldo para el caso en que la cámara detecta el cono pero el LIDAR no entrega un rango confiable. En el TurtleBot4, el RPLIDAR se encuentra en un plano distinto al de la cámara; si el cono queda por debajo de ese plano, el haz puede pasar por encima del cono y medir la pared de fondo. En ese caso, la estimación métrica basada en LIDAR ubica falsamente al cono sobre una celda ocupada. Para no descartar una detección visual válida, la misión entra en un modo de servoing visual: emite micro-objetivos cortos en la dirección angular del cono observada por la cámara. Cada micro-objetivo sigue pasando por el mismo validador geométrico contra el mapa inflado, por lo que se conserva la restricción de no publicar objetivos que atravesasen paredes.

VI-D. Máquina de estados de misión

La misión se diseñó como una FSM de supervisión. A diferencia del navegador, esta máquina no controla velocidades: decide cuándo buscar, cuándo aceptar una detección, cuándo enviar un objetivo y cuándo volver a intentar. La lógica final incluye dos caminos hacia el cono. Si la fusión cámara-LIDAR estima una posición confiable y navegable, la misión envía un objetivo de aproximación al navegador. Si el LIDAR no entrega un rango confiable o ubica el cono sobre una pared por medir el fondo, la misión entra en `SERVO_TO_CONE`: genera micro-objetivos sobre la dirección visual del cono,

siempre validados contra el mapa inflado. La Fig. 4 muestra el flujo conceptual actualizado.

Si el objetivo falla, queda bloqueado o el cono se pierde durante la aproximación, la misión vuelve a búsqueda o reintenta con una nueva estimación. En todos los casos, el control de bajo nivel queda dentro del navegador: la misión no publica velocidades, sino objetivos validados.

VII. ANÁLISIS DE RESULTADOS

El sistema se evaluó en tres entornos, en orden creciente de realismo: primero en simulación con el TurtleBot3 en el mundo de la casa (Partes A y B), después contra rosbags grabados con el TurtleBot4 en el laberinto real (Partes A, B y C, sin robot), y finalmente sobre el TurtleBot4 físico en el laboratorio (Partes A, B y C).

VII-A. TurtleBot3 en simulación: la casa (Partes A y B)

Para la Parte A se ejecutó FastSLAM recorriendo la casa con el robot, usando únicamente la odometría ruidosa de `/calc_odom`. Los primeros mapas presentaban inconsistencias, por lo que fue necesario diagnosticar el origen del error. El primer problema era de ROS y no del algoritmo: `/calc_odom` se publica con QoS `BEST_EFFORT`, y nuestra suscripción con la política por defecto perdía mensajes, así que el filtro recibía una odometría con agujeros. El segundo fue más interesante: la casa tiene salas casi cuadradas, y en una sala simétrica el scan matching puede “convencerse” de que el robot está rotado 90°, porque las paredes producen hipótesis geoméricamente similares. Cada tanto la mejor partícula saltaba a esa hipótesis y el mapa quedaba quebrado. El problema se resolvió acotando cuánto puede corregir el scan matching alrededor de la predicción de odometría: se mantiene la corrección fina, pero el salto de 90° deja de ser alcanzable. También se identificó que el yaw del sensor venía con un valor por defecto pensado para el TurtleBot4, que en el TurtleBot3 introducía una rotación de 90°; con `sensor_yaw=0` quedó resuelto. Con esos tres arreglos se obtuvo un mapa consistente y utilizable para la navegación de la Parte B.

Para la Parte B, con el mapa listo, se evaluó la navegación completa. La Fig. 5 muestra una situación representativa de navegación robusta: detección de un obstáculo sobre la

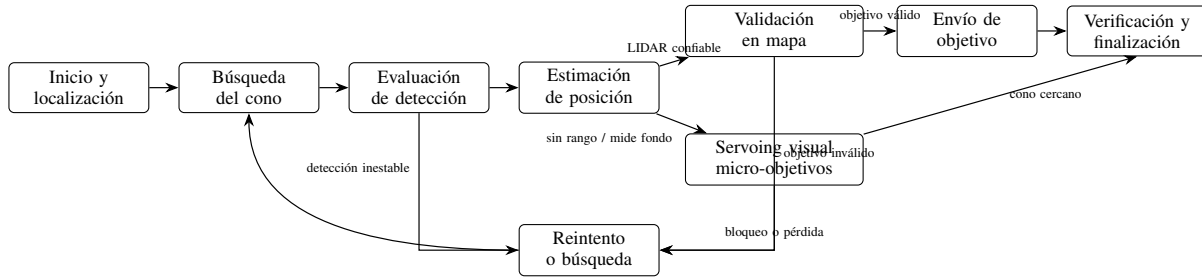


Figura 4. FSM de misión para el robot real. La misión transforma detecciones visuales en objetivos solo después de estimar su posición y validarla contra el mapa. Si la fusión cámara-LIDAR es confiable, se envía un objetivo de aproximación al navegador; si el LIDAR no entrega un rango confiable o mide la pared de fondo, se activa `SERVO_TO_CONE`, que avanza mediante micro-objetivos validados contra el mapa inflado.



(a) Detección de obstáculo en trayectoria.



(b) Replanteo de trayectoria.

Figura 5. Navegación en el entorno de casa simulado. La figura muestra dos momentos de una situación de bloqueo parcial: detección de un obstáculo en el camino y replanteo de la trayectoria para evitarlo.

trayectoria y replanteo del camino para evitarlo. Además de los tests unitarios del paquete (71 en total), se armó un conjunto de escenarios: objetivos en línea recta, objetivos que exigen un giro de 90° , objetivos cercanos a una pared y un objetivo deliberadamente inalcanzable. En los casos alcanzables el robot llegó al objetivo, y ante un caso bloqueado respondió de forma segura: en vez de insistir contra el obstáculo, agotó sus intentos de replanificación, declaró el bloqueo con la razón visible en `/nav_debug` y se quedó quieto publicando velocidad cero. En la demo final se encadenaron objetivos: el robot alcanzó dos objetivos consecutivos, replanificando al recibir cada objetivo nuevo, y ante un tercero imposible se detuvo de forma segura.

VII-B. TurtleBot4 sobre rosbags (Partes A, B y C)

Antes de la validación en laboratorio se evaluó todo lo que podía probarse sin ejecutar el robot en vivo, usando dos rosbags grabados con el TurtleBot4 en el laberinto físico. Un rosbag es una reproducción offline de tópicos ROS registrados

durante una corrida real: permite volver a procesar sensores reales, pero no valida el lazo cerrado de movimiento. Se usaron dos registros: uno de mapeo, con 23 minutos de LIDAR y odometría, y uno de percepción, con unos 15 minutos de cámara OAK-D y alrededor de 27 000 frames de 250×250 .

Para la Parte A, el mismo FastSLAM de la casa corrió sobre el bag de mapeo sin cambios de algoritmo y produjo un primer mapa completo del laberinto, con el que se validaron la localización y la navegación offline (el mapa definitivo se regeneró en vivo en el laboratorio; Sección VII-C). La principal dificultad en esta etapa fue computacional: con la configuración de máxima calidad no llegábamos a procesar los scans al ritmo del bag. Acelerar los kernels con Numba bajó el tiempo por actualización de 851 ms a 45 ms, una mejora cercana a $19\times$, lo que permitió ejecutar la configuración de mayor calidad en tiempo real en vez de reproducir el bag en cámara lenta.

Para la Parte B, la validación de localización sobre el bag permitió identificar una diferencia clave del porteo: el LIDAR

del TurtleBot4 está montado rotado $+90^\circ$ respecto de la base, y nuestro localizador asumía el scan alineado, como en el TurtleBot3. Lo confirmamos localizando globalmente el primer scan del bag y propagando la pose solo con odometría: con la corrección de $+90^\circ$ el match entre scan y mapa se mantiene en ~ 0.99 a lo largo de 1.8 m de recorrido, mientras que con offset cero cae a 0.79 y con -90° a 0.56. De ahí salieron los parámetros `scan_yaw_offset` y `scan_x_offset` que después usamos en el robot real. Además, se verificó el comportamiento del navegador en lazo cerrado con un mini-simulador cinemático que integra `/cmd_vel` y devuelve la pose, sin Gazebo: la secuencia `PLANNING` $\rightarrow$ `FOLLOWING` $\rightarrow$ `ALIGNING` $\rightarrow$ `REACHED` se completó con el robot a 0.12 m del goal.

Para la Parte C, el bag de percepción funcionó como dataset de calibración del detector. Lo que separa al cono de los distractores del laberinto no es el matiz sino la saturación: el cono es un rojo muy saturado ($S \approx 207$) y las barreras beige y de madera son naranjas lavados ($S \approx 66$). Subir el piso de saturación de 120 a 160 llevó la precisión de 0.74 a 0.87. El rango de matiz, en cambio, conviene dejarlo angosto (rojo puro): ensancharlo hacia el naranja mete distractores saturados y arruina la precisión. Con los umbrales finales barrimos 1798 frames del bag y obtuvimos 111 detecciones publicables (6.2% de los frames), con cero falsos positivos revisados a mano y rangos estimados entre 0.36 y 2.1 m (Fig. 6). Para dimensionar la mejora: con los umbrales iniciales teníamos unos 50 falsos positivos por reflejos y aristas rojizas.

Para pasar de la detección en imagen a una coordenada de mundo usamos fusión con LIDAR: el bearing de la cámara indexa el scan y de ahí sale el rango métrico. Validar esta geometría requería un criterio adicional: como el robot está casi quieto en tramos del bag, hasta una geometría equivocada hace que las estimaciones “agrupen”. Por eso usamos un criterio físico independiente de la pose: si de verdad estamos midiendo el cono, el tamaño del blob tiene que cumplir $\sqrt{\text{área}} \propto 1/\text{rango}$ (más cerca, más grande). Con el offset de indexación correcto (-90° , consistente con el montaje del LIDAR) la correlación da entre $+0.74$ y $+0.81$ y las posiciones estimadas del cono agrupan con ~ 1.6 cm de dispersión; con los offsets equivocados la correlación da negativa y las estimaciones se estiran en arcos que siguen al robot. También se comparó el uso del rayo más cercano contra la mediana local del scan. La mediana podía corresponder a la pared ubicada detrás del cono, por lo que se mantuvo el criterio del rayo más cercano.

Por último, se evaluó la máquina de estados de misión de punta a punta con percepción simulada, en dos escenarios. Con el cono en una celda libre, la misión llega a `DONE` emitiendo un único goal. Con el cono plantado sobre una pared — el caso “lo veo a través de un hueco” — la validación geométrica rechaza el objetivo y no se publica ningún goal hacia la pared. Un monitor aparte verificó en ambos casos que todos los `/goal_pose` cayeran en celda libre. También se evaluó un escenario sintético de obstáculo tipo silla delante del robot: el navegador hace un recovery acotado y termina en bloqueo

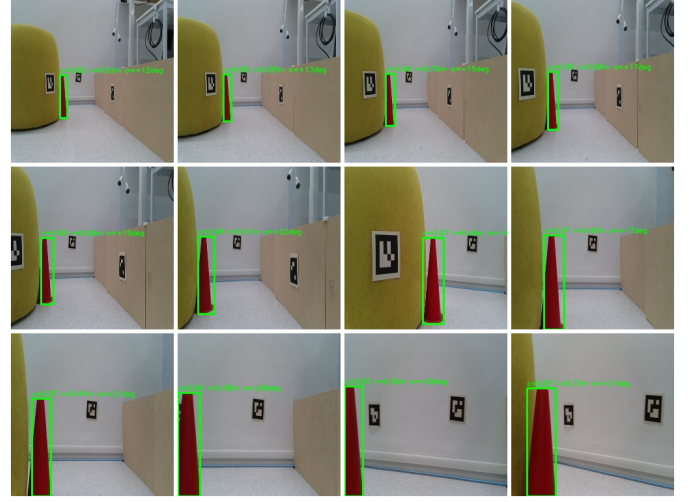


Figura 6. Detecciones del cono sobre el rosbag de percepción, anotadas con bounding box, confianza, rango y bearing.

seguro con la razón visible en `/nav_debug`, sin quedarse ciclando ida y vuelta.

VII-C. TurtleBot4 en el laboratorio (Partes A, B y C)

La validación en laboratorio permitió cerrar el ciclo completo sobre el TurtleBot4 real. Primero se obtuvo un mapa del laberinto y se respaldó la navegación sobre ese mapa. Luego se completó la misión visual del cono de punta a punta: el robot localizó, patrulló waypoints, realizó giros de búsqueda, detectó el cono, validó objetivos de aproximación y finalizó la misión frente al objeto. Las corridas se resumen en las figuras de esta sección y se complementan con registros audiovisuales [5].

En la Parte A se recorrió el laberinto con el TurtleBot4 y se ejecutó FastSLAM en vivo, obteniendo un mapa real de 500×500 celdas a 0.03 m por celda. Ese mapa se usó luego como base para localización, planificación y validación de misión. La Fig. 7 muestra el mapa resultante junto con una trayectoria planificada sobre él.

La Parte B quedó respaldada en laboratorio después de resolver tres diferencias prácticas respecto de la simulación. Primero, el robot no se movía porque el navegador publicaba en `/cmd_vel` global, mientras que el TurtleBot4 escucha en `/tb4_0/cmd_vel`; se corrigió mediante remapeo en el launch. Segundo, la cadena TF no cerraba porque el localizador tenía fijado el frame de odometría de la simulación (`calc_odom`) y el TB4 usa `odom`; se corrigió autodetectando el frame del mensaje. Tercero, los tópicos de TF del robot también están namespaced, por lo que fue necesario remapearlos para que RViz mostrara la corrección del MCL. Con esos arreglos, la MCL convergió a un spread de ~ 0.09 m y el robot pudo planificar sobre el laberinto. La trayectoria de la Fig. 7 no busca únicamente el camino más corto: aprovecha el mapa de costos inflado para mantener separación respecto de paredes y obstáculos, lo que reduce el riesgo de trayectorias pegadas a regiones no transitables.



Figura 7. Mapa del laberinto real y planificación sobre ese mapa. El mapa fue generado con FastSLAM durante la sesión de laboratorio y luego utilizado para localización, validación de misión y planificación; la trayectoria mantiene separación respecto de paredes y obstáculos de acuerdo con el mapa de costos inflado.

También se observó un caso representativo de obstáculo fino no consolidado en el mapa estático. Una silla ubicada en el recorrido no aparecía de forma estable en la grilla de ocupación: sus patas generaban retornos finos e intermitentes, por lo que el bloqueo no se integraba como una pared clara. Durante la navegación, el robot intentó avanzar por la zona, detectó bloqueos locales, retrocedió y reintentó. Luego de varios intentos, la lógica de progreso y recuperación forzó un replanteo que permitió rodear la silla y continuar la navegación. Este caso, registrado en video, muestra que el sistema no depende únicamente del mapa estático: combina planificación global, detección local y monitoreo de progreso para recuperarse ante obstáculos no modelados [5].

En la Parte C, las primeras pruebas de laboratorio permi-

tieron identificar una cadena de causas que impedía cerrar la misión. La cámara detectaba el cono, pero la misión no llegaba a producir una navegación efectiva hacia él. Esas causas se corrigieron antes de la validación final:

1. **Dependencia entre localización y movimiento.** La MCL sólo actualiza el peso de las partículas cuando el robot se mueve. En la primera versión, la misión esperaba convergencia en `LOCALIZE` sin generar movimiento, por lo que el spread de partículas podía quedar detenido en el valor inicial. La solución operativa fue iniciar la búsqueda después de obtener convergencia mediante movimiento de escaneo, y la misión final quedó preparada para avanzar sólo con localización estable.
2. **Waypoints inconsistentes con el mapa real.** Los waypoints de búsqueda iniciales estaban generados sobre un mapa anterior del rosbag. Al usarlos sobre el mapa real, varios caían fuera del espacio libre y eran rechazados por el validador. Se regeneraron los waypoints sobre el mapa real del laboratorio.
3. **Inflación excesiva del mapa de costos.** Con la inflación inicial, los pasillos angostos del laberinto quedaban cerrados en el costmap. Un barrido sobre el mapa mostró que bajar el margen total a 0.20 m recuperaba conectividad en el laberinto. Se unificó ese valor en el navegador, el validador de misión y el generador de waypoints.

Con estas correcciones, la misión completa se ejecutó de punta a punta en una corrida real de laboratorio. El robot comenzó patrullando waypoints, llegó a un sector cerrado, realizó un giro de búsqueda de 360° con la cámara, no encontró el cono y continuó hacia el siguiente waypoint. Al salir de ese sector, detectó el cono durante la navegación, estimó un objetivo de aproximación y terminó frente a él. La secuencia se resume en la Fig. 8 y corresponde al registro audiovisual end-to-end de esa corrida [5].

La Fig. 9 aporta una evidencia cualitativa adicional de la percepción en laboratorio. En esa escena, el cono rojo ya se encuentra dentro del campo de visión del TurtleBot4 y es detectado por el módulo visual. Aunque la detección por sí sola no determina el movimiento del robot, esta evidencia muestra que la percepción del objetivo funcionó correctamente en el entorno real y que luego pudo integrarse con la validación geométrica y la navegación de misión.

La integración real reveló además una diferencia física entre cámara y LIDAR. El cono rojo era suficientemente bajo como para quedar por debajo del plano del RPLIDAR: la cámara lo observaba, pero el LIDAR medía la pared de fondo. En la versión anterior, esa medición ubicaba el cono sobre una celda ocupada y la misión lo rechazaba como si estuviera detrás de una pared. Para resolverlo se incorporó el estado `SERVO_TO_CONE`. Cuando la fusión cámara-LIDAR no entrega un rango confiable, la misión genera micro-objetivos cortos sobre el bearing visual del cono. Cada micro-objetivo se valida contra el mapa inflado antes de publicarse, por lo que el sistema conserva el comportamiento seguro frente a paredes. La Fig. 10 muestra el caso en que la estimación basada en

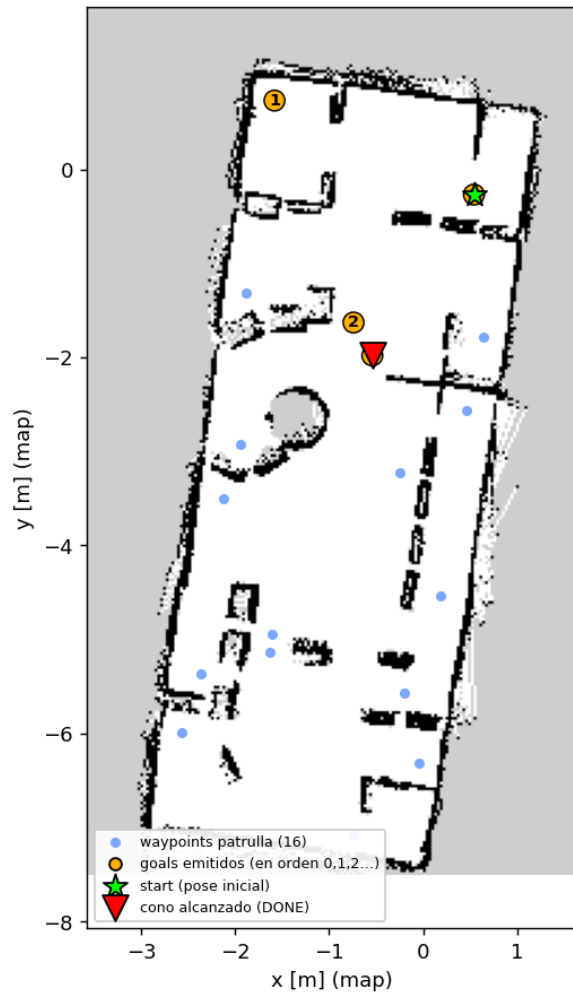


Figura 8. Corrida end-to-end de la misi3n real de b3squeda del cono. El TurtleBot4 patrulla waypoints, realiza b3squeda visual y completa la aproximaci3n al cono despu3s de detectarlo durante la navegaci3n.

LIDAR cae sobre una pared y dispara este fallback; en la validaci3n real, el mecanismo permiti3o confirmar el cono de cerca y finalizar la misi3n con estado DONE.

En conjunto, las Figs. 8 y 10 cubren dos caminos de 3xito: navegaci3n directa hacia el cono despu3s de b3squeda visual, y fallback de servoing visual cuando el LIDAR no entrega una estimaci3n confiable. En ambos casos, la misi3n alcanza el estado final de cono verificado. Los registros completos quedan disponibles como material complementario [5].

VII-D. Brecha sim-to-real

El pasaje al robot real introdujo diferencias que no aparecían en simulaci3n. Hubo que lidiar con los t3picos namespaces del TurtleBot4 (publicar en `/cmd_vel` global no mueve al robot: espera comandos en `/<ns>/cmd_vel`), con un 3rbol TF distinto al de simulaci3n, con el montaje del LIDAR a $+90^\circ$, con el QoS BEST_EFFORT de la odometría y con el compromiso de inflaci3n en pasillos angostos. Nada de esto es un error de los algoritmos: son diferencias sim-to-real



Figura 9. Evidencia cualitativa de percepci3n en laboratorio. El TurtleBot4 detecta visualmente el cono rojo durante una corrida real; esta detecci3n luego se integra con la validaci3n geom3trica y la navegaci3n de misi3n.

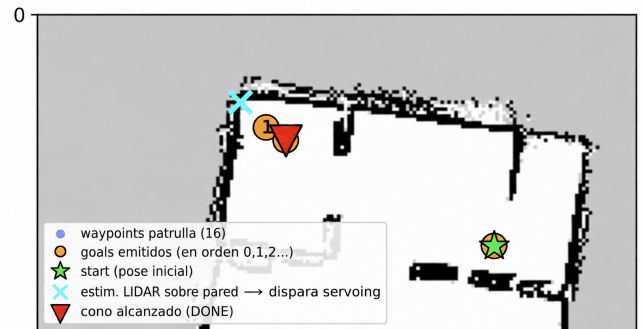


Figura 10. Detalle del fallback SERVO_TO_CONE. La c3mara detecta el cono, pero el LIDAR mide la pared de fondo y proyecta la estimaci3n sobre una celda ocupada; la misi3n responde con micro-objetivos sobre el bearing visual, validados contra el mapa inflado.

relevantes para evaluar el pasaje de simulaci3n a hardware real.

El trabajo offline con rosbags permiti3o resolver de antemano varias brechas de percepci3n y de geometría: el montaje del LIDAR a $+90^\circ$ lo descubrimos dos veces por caminos independientes (la consistencia scan-mapa de la localizaci3n y el criterio $\sqrt{\text{3rea}} \propto 1/\text{rango}$ de la fusi3n c3mara-LIDAR), y los umbrales HSV calibrados sobre el bag detectaron el cono en el laboratorio sin retoques.

Pero un bag es una grabaci3n: no reacciona a los comandos del robot. Un remapeo incorrecto de `/cmd_vel` no se manifiesta hasta que el robot debe ejecutar movimiento real. La dependencia entre movimiento y convergencia de la MCL tampoco aparece de la misma forma contra un bag, porque en el registro el robot ya est3 movi3ndose. Y la inflaci3n que

sellaba los pasillos sólo se manifiesta cuando un planificador intenta atravesarlos con el mapa real.

La validación offline permitió llegar al laboratorio con varios componentes ya depurados, especialmente percepción, geometría y localización. Sin embargo, la validación final mostró que el cierre de la misión exigía ejecutar el lazo completo sobre el robot físico. En esa etapa aparecieron diferencias propias del hardware, como la altura relativa entre el cono y el plano del RPLIDAR, que motivó el fallback de servoing visual.

VIII. LIMITACIONES

La solución presenta limitaciones conocidas. En primer lugar, la localización requiere una pose inicial razonable y suficiente movimiento para que las partículas converjan. En segundo lugar, la inflación de 0.20 m en el laberinto real es un compromiso: permite abrir pasillos angostos, pero reduce el margen físico frente a paredes. En tercer lugar, la detección de cono basada en HSV puede depender de iluminación, saturación y presencia de distractores cromáticos. En cuarto lugar, los obstáculos finos o con poca superficie, como patas de una silla, pueden no quedar consolidados de forma estable en el mapa estático. En esos casos, el sistema depende de la detección local, el monitoreo de progreso y la recuperación del navegador para evitar quedar bloqueado.

También existe una limitación geométrica propia de la configuración sensorial: cámara y LIDAR no observan exactamente el mismo volumen del espacio. En el laboratorio, el cono podía ser visible para la cámara pero quedar por debajo del plano del RPLIDAR, haciendo que el rango asociado al bearing visual correspondiera a la pared de fondo. El estado `SERVO_TO_CONE` resuelve este caso en la configuración probada, pero mantiene la necesidad de validar cada micro-objetivo contra el mapa inflado.

Por último, existen limitaciones operativas. La computadora del laboratorio puede tener diferencias de entorno, paquetes ROS 2 no instalados o problemas de display gráfico. Por ese motivo, las pruebas reales se complementaron con videos, capturas y logs de ejecución.

IX. CONCLUSIONES

Se desarrolló una arquitectura modular para abordar el ciclo completo de mapeo, localización, navegación y misión real en un robot móvil. La solución eligió una representación de mapa grillado, implementó localización probabilística mediante partículas, planificación global con A* y seguimiento de trayectoria con estados de navegación. Además, incorporó manejo conservador de obstáculos no mapeados y una capa de depuración mediante `/nav_debug`.

Para la misión real, se integró percepción de conos rojos y una FSM que supervisa la búsqueda sin asumir que una detección visual implica un camino directo libre. La validación geométrica de goals contra el mapa permite responder al caso de conos visibles a través de paredes o huecos. El comportamiento conservador del sistema resultó importante en la misión real: ante objetivos no validados, la arquitectura

priorizó no publicar movimientos inseguros antes que avanzar hacia una zona potencialmente no transitable.

El pasaje al TurtleBot4 real mostró que las pruebas en simulación y con rosbags reducen riesgos, pero no reemplazan la prueba cerrada sobre hardware. En laboratorio se obtuvo un mapa real utilizable, se navegó sobre ese mapa y se completó la misión visual del cono. La prueba final permitió cerrar el ciclo completo: localización, búsqueda, detección, validación de objetivos, navegación y verificación frente al cono. Además, el caso del cono bajo el plano del LIDAR motivó un fallback de servoing visual que conservó la validación geométrica de seguridad.

REFERENCIAS

- [1] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [2] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge University Press, 2006.
- [3] Open Robotics, “ROS 2 Documentation.” [Online]. Available: <https://docs.ros.org/>
- [4] M. Bianchi, J. Szterensus, J. C. Giner Pulero y V. Arbelaiz Alberti, “robot_maze_runner,” repositorio de código del proyecto, 2026. [Online]. Available: https://github.com/martinbianchi0/robot_maze_runner/tree/main
- [5] M. Bianchi, J. Szterensus, J. C. Giner Pulero y V. Arbelaiz Alberti, “Videos complementarios de laboratorio: misión end-to-end, búsqueda visual y recuperación ante obstáculo no mapeado,” carpeta de Google Drive, 2026. [Online]. Available: https://drive.google.com/drive/folders/11U41DnGkrGyjTxAmLq0Jf_iL1qgs2lnS